

Chapter 31. Causal Inference Without Experiments

Difference-in-differences, matching, regression discontinuity, synthetic control and instruments, each marked against a known true effect.

Draft chapter

Contents

Chapter 31. Causal Inference Without Experiments

Why this matters

In plain English

31.1 The data, and the three true answers

31.2 Why the two obvious comparisons mislead

31.3 Difference-in-differences

31.4 The assumption: parallel trends

31.5 Matching and propensity scores

31.6 Regression discontinuity: let the rule do the randomizing

31.7 Synthetic control

31.8 Instrumental variables: the idea

31.9 How much to trust each answer

Common mistakes and how to spot them

In the real world: the price rise nobody wanted to measure

Tools

The project: measure the regional price change

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 31. Causal Inference Without Experiments

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: say what a causal claim would mean when nobody randomized anything · spot why the two obvious comparisons (before-and-after, and treated-versus-untreated) usually mislead · estimate an effect with difference-in-differences, by hand and as a regression · test the parallel-trends assumption it rests on · build a comparison group with matching and propensity scores, and check the balance you achieved · use a threshold in a business rule as a natural experiment (regression discontinuity) · build a synthetic control from untreated groups · recognize a valid instrumental variable, and why you'll rarely have one · judge how much to trust each method, and write the claim up with its assumptions attached.

Before you start: Chapter 30 (confidence intervals, effect sizes, experiment design) and Chapter 22 (correlation is not causation). Chapter 19's regression mechanics help; Chapter 17's pandas is enough to follow the code.

Time needed: 10–14 hours, spread over two weeks.

Tools: Python 3.12 with pandas, numpy, scipy, statsmodels, and matplotlib.

Practice data: three datasets built by `generate_ch31_data.py` in the companion folder: 24 months of orders for Riverstone's four sales regions, 240 key accounts with a quarterly-review program, and 60,000 orders around the free-delivery threshold. Each one was generated with a **known true effect**, so every method in this chapter can be marked against the right answer.

Why this matters

Chapter 30's tools need a coin flip. Most business questions never get one:

- "What did the 6% price rise in the North region actually cost us in volume?" The price rose for every North customer on the same day. There is no control group, and no going back.
- "Do quarterly business reviews grow accounts?" The accounts that got them were chosen because they were big and growing.
- "Is free delivery above ₹25,000 bringing customers back?" The threshold applies to everyone.
- "Did the new warehouse cut delivery times?" It opened once, in one place.

You can't randomize any of these, and you still have to answer them, because somebody is going to act on the answer. The methods in this chapter get closer to a causal answer than "revenue went up after we did it", by finding a comparison that stands in for the experiment you couldn't run. They are

also what economists, policy analysts, and product data scientists spend most of their time on, and a favorite interview topic: "you can't A/B test this. How would you measure it?"

None of them is magic. Each replaces randomization with an **assumption**, and the discipline is to state the assumption, test what you can, and say what would break it.

In plain English

A causal claim is a comparison with a world that didn't happen.

"The price rise cost us 8% of our order volume" means: North's volume with the price rise, against North's volume in the same months *without* it. The second half doesn't exist. Every method here is a way of building a stand-in for it, called the **counterfactual**.

- **Before-and-after** uses North's own past as the stand-in. It fails whenever anything else changed at the same time: the season, the economy, a competitor.
- **Difference-in-differences** uses the other regions' change over the same months to measure "anything else", and subtracts it. The assumption is that North *would have moved like them* if nothing had happened.
- **Matching** builds the stand-in customer by customer: for each account in the program, find an account that looked the same beforehand and didn't get it.
- **Regression discontinuity** uses a rule. An order of ₹24,900 and one of ₹25,100 are nearly identical, except one gets free delivery. Just either side of a threshold, the rule does the randomizing for you.
- **Synthetic control** builds a fake North out of a weighted blend of the other regions, chosen so the blend tracks North's history closely, and then compares the real North with its double.
- **Instrumental variables** find something that nudges the treatment without touching the outcome any other way, and use only the variation that nudge explains.

Every one of them is an argument about the counterfactual, not a calculation that settles the matter. That's why this chapter spends as much time on assumptions and checks as on formulas.

31.1 The data, and the three true answers

```
import numpy as np
import pandas as pd

panel = pd.read_csv("causal_data/region_month.csv", parse_dates=["month"])
qbr = pd.read_csv("causal_data/qbr_program.csv")
orders = pd.read_csv("causal_data/delivery_threshold.csv")

print(panel.head(3).to_string(index=False))
print(f"\npanel: {panel.shape[0]} region-months, {panel['region'].nunique()} regions, "
      f"{panel['month'].min():%b %Y} to {panel['month'].max():%b %Y}")
print(f"qbr: {len(qbr)} accounts, {qbr['in_qbr_program'].sum()} in the program")
print(f"orders: {len(orders):,} orders around the ₹25,000 threshold")
```

region	month	orders	revenue	avg_order_value	price_rise_active	is_north	is_after
West	2024-07-01	3125	77605843.75	24833.87	0	0	0
West	2024-08-01	3061	78519945.53	25651.73	0	0	0
West	2024-09-01	3419	85941317.41	25136.39	0	0	0

```
panel: 96 region-months, 4 regions, Jul 2024 to Jun 2026
qbr: 240 accounts, 68 in the program
orders: 60,000 orders around the ₹25,000 threshold
```

Because the data is generated, the chapter can do something no real dataset allows: mark each method's homework.

What happened	True effect	Method
North's list prices rose about 6% on 1 October 2025	order volume −8.0%	difference-in-differences, synthetic control
Some key accounts were given quarterly business reviews during 2025	revenue +7.0%	matching and propensity scores
Orders of ₹25,000 or more get free delivery	repeat-order rate +6.0 points	regression discontinuity

Keep those three numbers in mind. Every estimate below should be judged against them, and none of them lands exactly.

31.2 Why the two obvious comparisons mislead

Before and after

```
panel["log_orders"] = np.log(panel["orders"])
north = panel[panel["region"] == "North"]
before = north.loc[north["is_after"] == 0, "orders"].mean()
after = north.loc[north["is_after"] == 1, "orders"].mean()

print(f"North, 15 months before the price rise: {before:,.0f} orders a month")
print(f"North, 9 months after: {after:,.0f} orders a month")
print(f"change: {100 * (after / before - 1):+.1f}%")
```

```
North, 15 months before the price rise: 2,032 orders a month
North, 9 months after: 2,067 orders a month
change: +1.7%
```

North's volume *went up* after a price rise that truly cost it 8% of its orders. Two things hide the damage: the business is growing by about 1% a month, and the price rose on 1 October, which is the start of the festive season. A before-and-after comparison credits the price rise with both.

Treated against untreated

```
post = panel[panel["is_after"] == 1]
north_post = post.loc[post["region"] == "North", "orders"].mean()
others_post = post.loc[post["region"] != "North", "orders"].mean()
print(f"North after: {north_post:,.0f} orders a month")
print(f"Other regions after: {others_post:,.0f} orders a month")
print(f"difference: {100 * (north_post / others_post - 1):+.1f}%")
```

```
North after: 2,067 orders a month
Other regions after: 2,683 orders a month
difference: -23.0%
```

Worse. North is a smaller region than West or South and always was, so most of that gap is history, not the price rise. The comparison measures the wrong thing entirely.

These two failures have names worth knowing:

- **Confounding**: something else moved at the same time as the treatment (the season, the growth trend) and gets counted as the treatment's effect.
- **Selection**: the treated group differs from the untreated one for reasons that already affected the outcome (North was always smaller; the biggest accounts were chosen for the program).

The methods in this chapter are all attempts to remove one or both.

31.3 Difference-in-differences

Difference-in-differences (DiD) takes the change in the treated group and subtracts the change in an untreated group over the same period. Whatever moved both groups (season, growth, the economy) cancels out.

Work it as a two-by-two table of average log orders:

```

panel["group"] = np.where(panel["region"] == "North", "North (treated)", "Other regions")
panel["period"] = np.where(panel["is_after"] == 1, "after", "before")
cells = panel.pivot_table(index="group", columns="period", values="log_orders", aggfunc="mean")
cells = cells[["before", "after"]]
cells["change"] = cells["after"] - cells["before"]

print(cells.round(4).to_string())
did = cells.loc["North (treated)", "change"] - cells.loc["Other regions", "change"]
print(f"\ndifference-in-differences (log): {did:+.4f}")
print(f"as a percentage change in orders: {100 * (np.exp(did) - 1):+.1f}%")
print("true effect: -8.0%")

```

```

period      before  after  change
group
North (treated)  7.6156  7.6328  0.0171
Other regions    7.7475  7.8375  0.0900

difference-in-differences (log): -0.0729
as a percentage change in orders: -7.0%
true effect: -8.0%

```

How it works:

- Work in **logs**, so a difference is a percentage change and the "parallel trends" assumption is about growth rates rather than absolute numbers. `exp(effect) - 1` converts back.
- North's own change is small and positive. The other regions' change over the same months is larger and positive. The *gap between the two changes* is the estimate.
- The estimate is -7.0% against a true -8.0%. That's an honest result for 24 months of noisy data, and it's a different universe from the +1.7% that before-and-after suggested.

The same thing as a regression

The two-by-two table only works for one treated group and two periods. Written as a regression, DiD extends to many groups, many periods, and controls:

```

import statsmodels.formula.api as smf

panel["treated"] = (panel["region"] == "North").astype(int)
simple = smf.ols("log_orders ~ treated + is_after + treated:is_after", data=panel).fit()
print(simple.summary().tables[1])

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept          7.7475        0.045     171.944      0.000         7.658         7.837
treated           -0.1318        0.090     -1.463      0.147        -0.311         0.047
is_after           0.0900        0.074       1.224      0.224        -0.056         0.236
treated:is_after   -0.0729        0.147     -0.495      0.622        -0.365         0.219
=====

```

The coefficient on `treated:is_after` is exactly the number from the table: that interaction is difference-in-differences. Its confidence interval, though, is uselessly wide, because this specification makes the model explain every difference between regions and months with three dummies.

Adding **fixed effects** for region and month removes all of that variation, leaving the model to explain only what happens within a region relative to what happened to everyone that month:

```
fe = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
effect = fe.params["treated:is_after"]
ci = fe.conf_int().loc["treated:is_after"]
print(f"effect (log):      {effect:+.4f}")
print(f"95% CI:           {ci[0]:+.4f} to {ci[1]:+.4f}")
print(f"as a percentage:  {100 * (np.exp(effect) - 1):+.1f}%  "
      f"(CI {100 * (np.exp(ci[0]) - 1):+.1f}% to {100 * (np.exp(ci[1]) - 1):+.1f}%)")
print(f"R-squared: {fe.rsquared:.3f}")
```

```
effect (log):      -0.0729
95% CI:           -0.1064 to -0.0394
as a percentage:  -7.0%  (CI -10.1% to -3.9%)
R-squared: 0.991
```

Same estimate, a far tighter interval, and the true -8% sits comfortably inside it. **Region fixed effects** absorb "North is smaller"; **month fixed effects** absorb the festive season and everything else that hit all regions at once.

Watch out: four regions is not many. Standard errors in DiD should allow for the fact that observations within a region are related month to month, usually by **clustering** them: `fe = smf.ols(...).fit(cov_type="cluster", cov_kws={"groups": panel["region"]})`. With only four clusters, the usual formulas are unreliable, and the honest options are a wild cluster bootstrap, randomization inference, or a clearly stated caveat. If your DiD has a handful of groups, say so in the write-up rather than quoting a tidy p-value.

31.4 The assumption: parallel trends

DiD rests on one assumption: **without the price rise, North's orders would have moved like the other regions' orders**. That world doesn't exist, so it can't be tested. What can be tested is whether the two moved together *before* the change.

```
monthly = (panel.assign(group=np.where(panel["treated"] == 1, "North", "Others"))
             .pivot_table(index="month", columns="group", values="log_orders", aggfunc="mean"))
monthly["gap"] = monthly["North"] - monthly["Others"]
pre = monthly[monthly.index < "2025-10-01"]
post = monthly[monthly.index >= "2025-10-01"]

print("gap between North and the other regions (log orders)")
print(f" before the price rise: mean {pre['gap'].mean():+.4f}, sd {pre['gap'].std():.4f}")
print(f" after:                    mean {post['gap'].mean():+.4f}")
print(f" drift per month before the change: "
      f"{np.polyfit(range(len(pre)), pre['gap'], 1)[0]:+.5f}")
print(pre["gap"].round(3).tail(4).to_string())
```

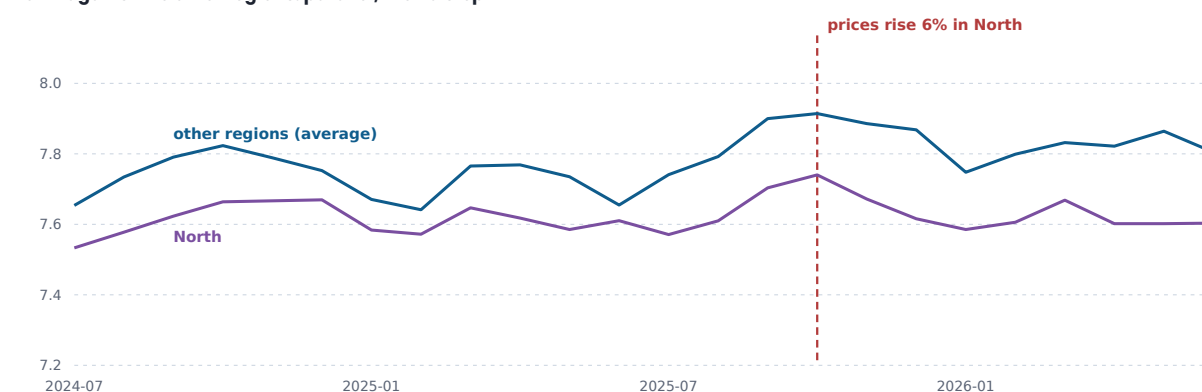
```

gap between North and the other regions (log orders)
before the price rise: mean -0.1318, sd 0.0446
after:                    mean -0.2047
drift per month before the change: -0.00175
month
2025-06-01    -0.044
2025-07-01    -0.170
2025-08-01    -0.182
2025-09-01    -0.196

```

Reading it: before the change, the gap between North and the rest has no trend worth the name: it wanders with a standard deviation of about 4.5% and drifts by -0.18% a month, which over fifteen months comes to less than a third of the effect being measured. After the change the gap drops by about 7 points and stays there. Flat-ish before, a step at the right moment, and a new level after is the pattern DiD needs. The last two pre-change months do sit low, which is why the next check tracks the gap over time instead of comparing two averages.

North against the other regions: parallel, then a step



Log of monthly orders. Before October 2025 the gap between the lines wanders without direction; after it, North sits about 7% lower and stays there. Difference-in-differences is the change in that gap: -7.0% (95% CI -10.1% to -3.9%). The true effect built into the data is -8% .

Figure 31.1 – Parallel before, a step at the change, parallel again after. This picture belongs in every DiD write-up.

A quarter-by-quarter view

A stricter look tracks the gap over time rather than averaging the whole period. Single months are too noisy for one region, so group them into quarters around the change, measured against the average gap in the fifteen months before it:

```

monthly["rel"] = ((monthly.index.year - 2025) * 12 + monthly.index.month - 10)
baseline = monthly.loc[monthly["rel"] < 0, "gap"].mean()          # the whole pre-change period
monthly["quarter"] = np.floor(monthly["rel"] / 3).astype(int)
view = (monthly.groupby("quarter")["gap"].mean() - baseline).round(3)
for quarter, value in view.items():
    when = "before" if quarter < 0 else "after"
    print(f"quarter {quarter:+d} ({when}): {value:+.3f} {'#' * int(abs(value) * 100)}")

```

```

quarter -5 (before): -0.016 #
quarter -4 (before): +0.010 #
quarter -3 (before): +0.040 ####
quarter -2 (before): +0.017 #
quarter -1 (before): -0.051 #####
quarter +0 (after ): -0.082 #####
quarter +1 (after ): -0.041 ####
quarter +2 (after ): -0.096 #####

```

The five quarters before the change sit within five points of the baseline, with no direction to them. Every quarter after it sits between four and ten points below. That's the pattern DiD assumes: no pre-trend, then a level shift. With one treated region and 24 months there isn't enough data for a formal event-study regression with confidence intervals, and pretending otherwise would be exactly the kind of tidy-looking mistake this chapter warns about.

Watch out: parallel trends can fail quietly. Common causes: the treated group was picked *because* it was already declining (Riverstone might have raised prices in North precisely because volume was soft); a different event hit only the treated group at the same time; or the groups have different seasonal patterns. Check what you can, and name what you can't: *"we assume the North region would have grown like the others, which held for the fifteen months we can see."*

31.5 Matching and propensity scores

Quarterly business reviews went to 68 of Riverstone's 240 key accounts. The naive comparison is spectacular and wrong:

```

qbr["log_2025"] = np.log(qbr["revenue_2025"])
qbr["log_2024"] = np.log(qbr["revenue_2024"])
naive = (qbr.groupby("in_qbr_program")
         .agg(accounts=("customer_id", "count"),
              revenue_2024=("revenue_2024", "mean"),
              revenue_2025=("revenue_2025", "mean"),
              growth_2024=("growth_2024", "mean")))
print(naive.round(3).to_string())
gap = qbr.groupby("in_qbr_program")["log_2025"].mean().diff().iloc[-1]
print(f"\nnaive difference in 2025 revenue: {100 * (np.exp(gap) - 1):+.1f}%")
print("true effect: +7.0%")

```

	accounts	revenue_2024	revenue_2025	growth_2024
in_qbr_program				
0	172	755520.916	824816.990	0.052
1	68	1225516.825	1420723.119	0.062

naive difference in 2025 revenue: +75.0%
true effect: +7.0%

The accounts in the program were already twice the size of the others *before* it started, and growing faster. Selection, in one table.

Matching answers a narrower question: for each account that joined, what happened to a similar account that didn't? A **propensity score** compresses "similar" into one number: the modelled probability of joining, given what was known beforehand.

```
score_model = smf.logit("in_qbr_program ~ log_2024 + growth_2024 + years_as_customer + C(segment)",
                        data=qbr).fit(dispatch=False)
qbr["propensity"] = score_model.predict(qbr)
print(qbr.groupby("in_qbr_program")["propensity"].describe()[["count", "mean", "min",
"max"]].round(3).to_string())
```

	count	mean	min	max
in_qbr_program				
0	172.0	0.255	0.032	0.686
1	68.0	0.354	0.130	0.802

The two groups overlap but sit in different parts of the range, which is exactly what a selected program looks like. Now match each participant to the closest non-participant by score:

```
treated = qbr[qbr["in_qbr_program"] == 1].copy()
controls = qbr[qbr["in_qbr_program"] == 0].copy()
pairs = []
available = controls.set_index("customer_id")["propensity"].to_dict()
for _, row in treated.sort_values("propensity", ascending=False).iterrows():
    if not available:
        break
    best = min(available, key=lambda cid: abs(available[cid] - row["propensity"]))
    if abs(available[best] - row["propensity"]) <= 0.05:          # caliper: refuse poor matches
        pairs.append((row["customer_id"], best))
        del available[best]                                       # match without replacement
matched_ids = [cid for pair in pairs for cid in pair]
matched = qbr[qbr["customer_id"].isin(matched_ids)]
print(f"matched pairs: {len(pairs)} of {len(treated)} participants")
print(f"unmatched participants: {len(treated) - len(pairs)} (no close enough comparison existed)")
```

```
matched pairs: 64 of 68 participants
unmatched participants: 4 (no close enough comparison existed)
```

Check balance before believing anything

Matching is only worth something if the matched groups now look alike on everything you matched on. The usual measure is the **standardized mean difference**: the gap in means divided by the pooled standard deviation. Under 0.1 is the common rule of thumb.

```
def smd(data, column):
    a = data.loc[data["in_qbr_program"] == 1, column]
    b = data.loc[data["in_qbr_program"] == 0, column]
    return (a.mean() - b.mean()) / np.sqrt((a.var(ddof=1) + b.var(ddof=1)) / 2)

rows = []
for column in ["log_2024", "growth_2024", "years_as_customer", "propensity"]:
    rows.append({"variable": column, "before matching": smd(qbr, column), "after matching":
smd(matched, column)})
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

variable	before matching	after matching
log_2024	0.635	0.002
growth_2024	0.082	-0.098
years_as_customer	0.239	0.106
propensity	0.683	0.033

Matching is only worth something if it balanced the groups

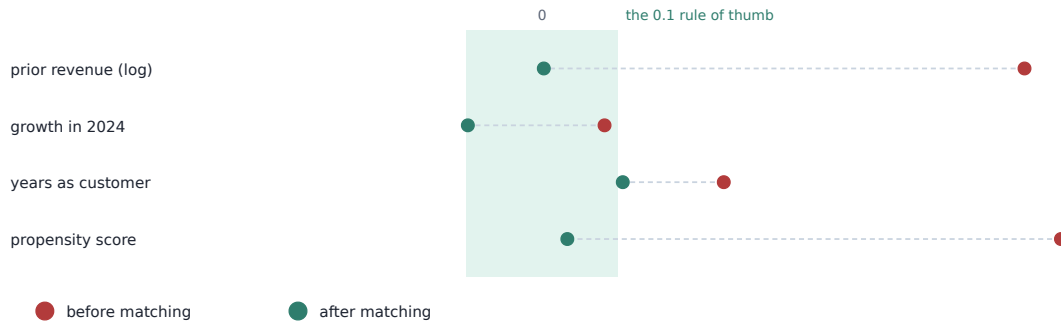


Figure 31.2 — Balance is the checkable part of matching. Report it, or the estimate means nothing.

The estimate

```
matched_effect = matched.groupby("in_qbr_program")["log_2025"].mean().diff().iloc[-1]
model = smf.ols("log_2025 ~ in_qbr_program + log_2024 + growth_2024 + years_as_customer",
               data=qbr).fit()
adjusted = model.params["in_qbr_program"]
ci = model.conf_int().loc["in_qbr_program"]

print(f"naive: {100 * (np.exp(gap) - 1):+.1f}%")
print(f"after matching: {100 * (np.exp(matched_effect) - 1):+.1f}%")
print(f"regression adjustment: {100 * (np.exp(adjusted) - 1):+.1f}% "
      f"(95% CI {100 * (np.exp(ci[0]) - 1):+.1f}% to {100 * (np.exp(ci[1]) - 1):+.1f}%)")
print("true effect: +7.0%")
```

```
naive: +75.0%
after matching: +7.1%
regression adjustment: +7.7% (95% CI +4.6% to +10.9%)
true effect: +7.0%
```

Both corrections land near the truth, and both are a world away from the naive +75%. Note what made this work: every variable that drove selection (size, growth, tenure) was **measured and available**. That is the assumption matching rests on, and it has a name.

Watch out: matching only fixes what you measured. The assumption is **no unmeasured confounding**: nothing you left out of the score influenced both joining and the outcome. Here it holds by construction, because the data was generated that way. In real life, the sales team's judgment about which accounts "had potential" is exactly the kind of unmeasured variable that ruins it. That's why matching is weaker evidence than a threshold rule or an experiment, and why sensitivity analysis (section 31.9) matters.

31.6 Regression discontinuity: let the rule do the randomizing

Riverstone gives free delivery on orders of ₹25,000 or more (Chapter 3). Does it bring customers back?

Comparing all orders above the threshold with all orders below it is hopeless: big orders come from big customers, who reorder anyway. But an order of ₹24,900 and one of ₹25,100 are placed by much the same kind of customer, in much the same mood. One gets free delivery and one doesn't, for a reason that has nothing to do with either. **Just around the threshold, the rule is as good as a coin flip.** That's **regression discontinuity (RD)**.

```
orders["centred"] = (orders["order_value"] - 25_000) / 1000    # in thousands of rupees from the cut-off
wide = orders[orders["centred"].abs() <= 5]
below = wide[wide["centred"] < 0]["repeat_within_90_days"]
above = wide[wide["centred"] >= 0]["repeat_within_90_days"]

print(f"orders within ₹5,000 of the threshold: {len(wide):,}")
print(f"just below: {100 * below.mean():.1f}% reorder within 90 days (n = {len(below):,})")
print(f"just above: {100 * above.mean():.1f}% (n = {len(above):,})")
print(f"raw gap: {100 * (above.mean() - below.mean()):+.1f} points")
```

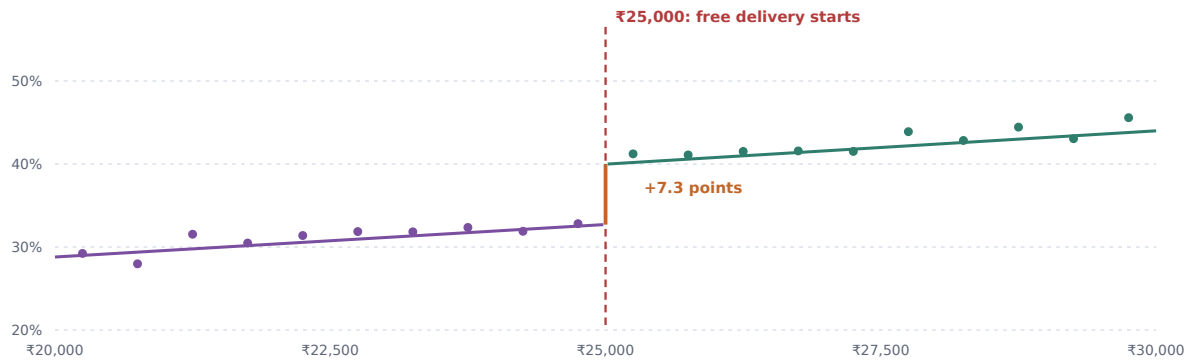
```
orders within ₹5,000 of the threshold: 46,054
just below: 31.4% reorder within 90 days (n = 22,958)
just above: 42.4% (n = 23,096)
raw gap: +11.0 points
```

The raw gap overstates things, because the chance of reordering rises with order value anyway. RD fits a line on each side and measures the **jump at the cut-off**, not the level:

```
rd = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
data=wide).fit()
jump = rd.params["free_delivery"]
ci = rd.conf_int().loc["free_delivery"]
print(f"jump at the threshold: {100 * jump:+.1f} points (95% CI {100 * ci[0]:+.1f} to {100 * ci[1]:+.1f})")
print(f"slope below the cut-off: {100 * rd.params['centred']:+.2f} points per ₹1,000")
print("true effect: +6.0 points")
```

```
jump at the threshold: +7.3 points (95% CI +5.6 to +8.9)
slope below the cut-off: +0.79 points per ₹1,000
true effect: +6.0 points
```

Free delivery at ₹25,000: the estimate is the size of the step



Each dot is the reorder rate for orders in a ₹500 band. The slope either side is the ordinary relationship between order size and reordering; only the jump at the line is caused by the rule. True effect built into the data: +6.0 points.

Figure 31.3 — The estimate is the size of the step, not the difference between the two sides.

Bandwidth: how close is close enough?

Narrow windows are more credible and noisier; wide ones are precise and lean more on the model. Report several:

```
rows = []
for bandwidth in (0.5, 1, 2, 3, 5):
    window = orders[orders["centred"].abs() <= bandwidth]
    fit = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
data=window).fit()
    low, high = fit.conf_int().loc["free_delivery"]
    rows.append({"bandwidth (₹000)": bandwidth, "orders": len(window),
        "estimate (pts)": 100 * fit.params["free_delivery"],
        "ci_low": 100 * low, "ci_high": 100 * high})
print(pd.DataFrame(rows).round(2).to_string(index=False))
print("true effect: +6.0 points")
```

bandwidth (₹000)	orders	estimate (pts)	ci_low	ci_high
0.5	5686	6.34	1.29	11.38
1.0	11331	8.30	4.75	11.84
2.0	22095	8.05	5.54	10.57
3.0	31502	7.65	5.57	9.72
5.0	46054	7.26	5.61	8.90

true effect: +6.0 points

Every window puts the jump between 6 and 9 points, and every interval contains the true 6. That stability across bandwidths is part of the evidence; an estimate that swings when the window changes is a warning, not a result.

The check that matters: can people move themselves across the line?

RD works only if nobody can precisely control which side of the threshold they land on. If customers nudge a ₹24,400 order up to ₹25,000 to get free delivery, the two sides stop being comparable, because the ones who nudged are the keen ones. The test is to look for **bunching** just above the cut-off:

```
bins = pd.cut(orders["centred"], bins=np.arange(-5, 5.5, 0.5))
density = orders.groupby(bins, observed=True).size()
print(density.tail(14).to_string())
just_below = density[[i for i in density.index if -1.0 <= i.left < 0]].sum()
just_above = density[[i for i in density.index if 0 <= i.left < 1.0]].sum()
print(f"\norders in the ₹1,000 below: {just_below:,}")
print(f"orders in the ₹1,000 above: {just_above:,}")
print(f"ratio: {just_above / just_below:.3f} (bunching would push this well above 1)")
```

```
centred
(-2.0, -1.5]    2620
(-1.5, -1.0]    2738
(-1.0, -0.5]    2837
(-0.5, 0.0]     2831
(0.0, 0.5]      2855
(0.5, 1.0]      2808
(1.0, 1.5]      2770
(1.5, 2.0]      2636
(2.0, 2.5]      2489
(2.5, 3.0]      2195
(3.0, 3.5]      2108
(3.5, 4.0]      1917
(4.0, 4.5]      1756
(4.5, 5.0]      1562

orders in the ₹1,000 below: 5,668
orders in the ₹1,000 above: 5,663
ratio: 0.999 (bunching would push this well above 1)
```

Smooth. In real data this check often fails: sales reps round orders up to hit thresholds, and expense claims cluster below approval limits. When it fails, RD is not available, and the bunching itself becomes the finding.

Watch out: RD only measures the effect at the cut-off. This estimate says what free delivery does for orders near ₹25,000. It says nothing about ₹5,000 orders, where the delivery cost is a much larger share of the order, or ₹200,000 ones, where nobody cares. That narrowness is the price of the credibility: a **local** answer you can trust more than a global one you can't.

31.7 Synthetic control

DiD compares North with the *average* of the other regions. But West is twice North's size and East is smaller; why should their average be the right stand-in?

Synthetic control builds a better one: a weighted blend of the untreated regions, with the weights chosen so the blend tracks the treated region as closely as possible *before* the change. Then it carries those weights forward and compares.

```

from scipy.optimize import minimize

wide_panel = panel.pivot_table(index="month", columns="region", values="log_orders")
pre_period = wide_panel[wide_panel.index < "2025-10-01"]
donors = ["West", "South", "East"]

def loss(weights):
    blend = pre_period[donors].to_numpy() @ weights
    return float(np.mean((pre_period["North"].to_numpy() - blend) ** 2))

start = np.repeat(1 / len(donors), len(donors))
bounds = [(0, 1)] * len(donors)
constraint = {"type": "eq", "fun": lambda w: w.sum() - 1}      # weights are a blend: non-
negative, summing to 1
best = minimize(loss, start, bounds=bounds, constraints=[constraint], method="SLSQP")
weights = dict(zip(donors, best.x.round(3)))
print("synthetic North =", weights)

synthetic = wide_panel[donors].to_numpy() @ best.x
fit_before = np.sqrt(np.mean((pre_period["North"].to_numpy() - synthetic[:len(pre_period)]) ** 2))
after = wide_panel.index >= "2025-10-01"
gap_after = (wide_panel["North"].to_numpy() - synthetic)[after].mean()
print(f"pre-change fit (root mean squared error, log orders): {fit_before:.4f}")
print(f"average gap after the change: {gap_after:+.4f} log points "
      f"= {100 * (np.exp(gap_after) - 1):+.1f}%")
print("true effect: -8.0%")

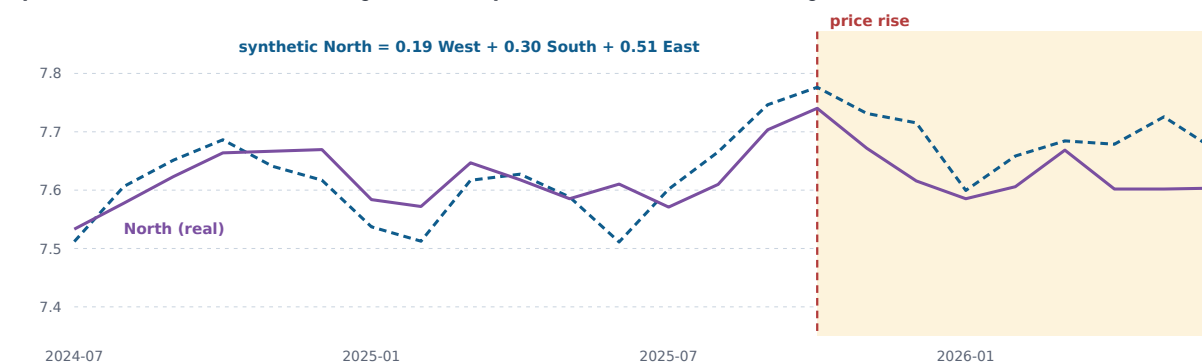
```

```

synthetic North = {'West': np.float64(0.187), 'South': np.float64(0.303), 'East':
np.float64(0.509)}
pre-change fit (root mean squared error, log orders): 0.0429
average gap after the change: -0.0685 log points = -6.6%
true effect: -8.0%

```

Synthetic North: a blend of the other regions, built only from the months before the change



The weights were chosen to track North for the fifteen months before October 2025 (root mean squared error 0.043 in log orders). Everything in the shaded area is the estimate: an average gap of -6.6%, against a true effect of -8%.

Figure 31.4 — The synthetic region is built only from pre-change data. Everything after the line is the estimate.

How it works and what to watch:

- The weights are non-negative and sum to 1, so the synthetic region is a genuine blend, not an extrapolation. That restraint is what keeps the method honest.
- **Judge it by the pre-change fit first.** A synthetic control that doesn't track the treated unit before the change has no claim to track it after.

- With four regions there are only three donors, so the method is doing little more than a weighted DiD. Synthetic control earns its keep with dozens of candidate donors: shops, cities, states, or customers.
- **Inference** is done by pretending each untreated unit was treated (a **placebo test**) and asking how unusual the real gap looks against theirs. Exercise 11 does this.

Watch out: donors must be untouched. If the price rise spilled into a neighboring region (customers near the border ordering from West instead), West is contaminated and the estimate shrinks toward zero. Ask, every time: could the treatment have affected the comparison group? The same question ruins many DiD studies, and it has a name: **interference** between units.

31.8 Instrumental variables: the idea

Sometimes nothing above is available, because the thing you care about is chosen by the people you're studying. "Do site visits by a sales rep increase orders?" Reps visit the customers most likely to buy, and no threshold or clean before-and-after exists.

An **instrumental variable** is something that shifts the treatment but affects the outcome *only* through it. Suppose reps visit customers in their own city far more often, because travel time decides the day's route. Distance from the rep's office then nudges the number of visits without, in itself, making a customer buy more plastic crates. The method uses only the variation in visits that distance explains, and discards the rest, which is the part contaminated by the rep's judgment.

A valid instrument needs three things:

Requirement	What it means	For "distance to the rep's office"
Relevance	it really moves the treatment	nearby customers do get more visits: check it, don't assume it
Exclusion	it affects the outcome <i>only</i> through the treatment	distance must not affect orders any other way, which is doubtful: nearby customers may also get faster delivery
Independence	it isn't driven by something that also drives the outcome	offices were placed near big customers, so distance is already related to size

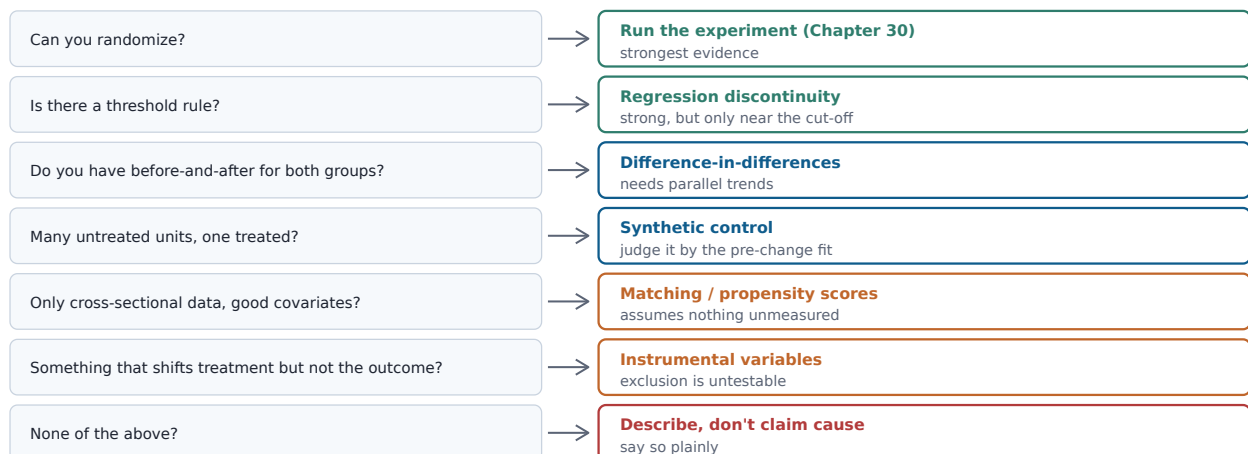
The last two are assumptions, and cannot be tested. The estimation itself is routine (two-stage least squares: predict the treatment from the instrument, then use the prediction), and it is never the hard part. **Finding a defensible instrument is.** Classic examples earned their fame because they're rare: distance to the nearest college for the effect of education on earnings, quarter of birth for years of schooling, rainfall for economic shocks.

For an analyst, the useful skill is recognizing the shape of the argument: "*we are using only the part of the variation caused by something unrelated to the outcome*". That lets you read a study, ask the exclusion question in a meeting, and sometimes spot an instrument hiding in your own systems: a policy that applied to some customers by an accident of timing, a system migration that happened alphabetically, an outage that hit one region's warehouse.

31.9 How much to trust each answer

Method	The comparison it builds	Key assumption	What you can check	Strength of evidence
Before and after	the group's own past	nothing else changed	other series over the same period	weak
Treated vs untreated	another group, right now	the groups were alike already	pre-period levels and trends	weak
Difference-in-differences	the untreated group's change	parallel trends	pre-trends, event study, placebo periods	moderate to strong
Matching / propensity	similar-looking untreated units	no unmeasured confounding	balance, overlap, sensitivity analysis	moderate
Regression discontinuity	units just either side of a rule	no precise manipulation of the running variable	bunching, covariate smoothness, bandwidth stability	strong, but local
Synthetic control	a weighted blend of untreated units	the blend tracks the treated unit for the right reasons	pre-change fit, placebo units	moderate to strong
Instrumental variables	variation caused by the instrument	relevance, exclusion, independence	relevance only	strong if the instrument is valid, which is rare
Randomized experiment (Chapter 30)	a coin flip	the randomization worked	sample-ratio and balance checks	strongest

Which method, and what it costs you



Work down the list: the first row you can answer yes to is usually the strongest evidence available to you.

Figure 31.5 – Work down the list; the first row you can answer yes to is usually the strongest evidence available.

Sensitivity: how wrong could you be?

The question to ask of any observational estimate is not "is it right?" but "how strong would the thing I've missed have to be, to overturn this?"

```
naive_pct = 100 * (np.exp(gap) - 1)
adjusted_pct = 100 * (np.exp(adjusted) - 1)
print(f"quarterly reviews, unadjusted: {naive_pct:+.1f}%")
print(f"after controlling for size, growth and tenure: {adjusted_pct:+.1f}%")
print(f"share of the naive gap that was selection: {100 * (1 - adjusted_pct / naive_pct):.0f}%")
print("\nA confounder that explained as much as prior size did would wipe out what's left.")
```

```
quarterly reviews, unadjusted: +75.0%
after controlling for size, growth and tenure: +7.7%
share of the naive gap that was selection: 90%
```

A confounder that explained as much as prior size did would wipe out what's left.

Most of the apparent effect was selection. A missing variable of similar strength to the ones already in the model could account for the remainder, so the honest sentence is: *"about +7%, and that estimate depends on the sales team's choices being driven by things we can see."* Formal versions of this reasoning exist (Rosenbaum bounds, E-values); the informal version, said out loud, is already most of the value.

Writing up a causal claim

Four sentences, in this order:

1. **The estimate, with its interval and its unit.** *"North's order volume fell about 7% (95% CI 4% to 10%) after the price rise."*
2. **The comparison that produced it.** *"Measured against the other three regions over the same months, with region and month effects removed."*
3. **The assumption it rests on, and the check you ran.** *"This assumes North would otherwise have grown like the others; their trends ran parallel for the fifteen months before the change."*
4. **What would change your mind.** *"If something else hit North in October, such as a competitor or a lost distributor, this would overstate the price effect. I checked the sales log and found nothing, but it's the risk."*

Interview extra point. Asked "how would you measure something you can't A/B test?", name the method, then immediately name its assumption and the check: *"difference-in-differences, assuming parallel trends, which I'd test on the pre-period and show as a chart."* Chapter 69's method calls this pattern claim-plus-check, and it separates people who have read about these methods from people who have used them.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Before-and-after with no comparison group	The season, the trend, or a campaign gets credited to your change	Find an untreated group and use DiD
Comparing treated with untreated as they are	Differences that existed beforehand read as effects	Match, or use each group's <i>change</i>
Skipping the parallel-trends check	A DiD that measures a divergence already under way	Plot the pre-period; show the chart in the write-up
Reading a DiD from few clusters as precise	Tidy p-values from four regions	Cluster, then say plainly that the number of groups is small
Matching on variables measured after the treatment	The control variable is itself an effect; the estimate collapses	Only use what was known before treatment began
Not checking balance after matching	Matching that didn't work, reported as if it did	Standardized mean differences, before and after
Matching outside the region of overlap	Participants with no comparable control get matched anyway	A caliper, and report how many went unmatched
Treating matching as equal to an experiment	Confident causal language on shaky ground	Say "assuming nothing unmeasured drove selection"
RD with a manipulable threshold	Bunching just above the cut-off	Check the density; if it bunches, don't use RD
RD generalized beyond the cut-off	"Free delivery raises reorders by 7 points" for all orders	Say "for orders near ₹25,000"
One bandwidth, chosen after seeing results	An estimate the reader can't audit	Report a table of bandwidths
Synthetic control with a poor pre-change fit	A "control" that never tracked the treated unit	Judge the fit first; abandon the method if it's bad
Contaminated comparison groups	Effects that leak into the control and shrink the estimate	Ask whether the treatment could have touched the comparison
An instrument that affects the outcome directly	A causal claim resting on an untestable assumption nobody stated	State exclusion out loud; most instruments fail it
Causal language without the assumption	"The price rise cost us 7%" with nothing attached	Estimate, comparison, assumption, and what would change your mind
Chasing significance instead of magnitude	A statistically significant 0.4% effect driving a big decision	Chapter 30's rule: effect, interval, and sample size

In the real world: the price rise nobody wanted to measure

Riverstone's North region raised list prices 6% on 1 October 2025, to protect margin after a polymer price shock. In April 2026 the board asks the obvious question, and gets three answers in one meeting.

The regional manager says volume is up 1.7% since the rise, so customers clearly accepted it. The finance manager says North sells 23% less than the other regions, so the rise has done real damage. Both are reading the same table.

Meera's answer takes an afternoon. She builds the region-month panel, plots North against the other three, and shows that the lines ran parallel for fifteen months and then stepped apart in October. Her estimate is a 7% fall in order volume, with an interval from 4% to 10%. Multiplied by the price increase, margin is roughly flat: the rise paid for itself and no more.

Then the useful part of the meeting. The sales director asks what else changed in North in October. Meera has checked: no distributor was lost, no competitor opened a depot, the festive season affected all four regions alike. She also says what she can't rule out: with one treated region and four groups in total, the interval is wider than it looks, and a single unusual month moves it.

The decision is not "the price rise failed". It is to hold the new prices, and to run a proper experiment on the next change: a randomized price test across a sample of customers in two regions, designed with Chapter 30's arithmetic, so that next year's version of this meeting has a coin flip behind it instead of an argument.

What she tells the board: *"Volume fell about 7%, not rose 1.7%. The 1.7% was the festive season and our overall growth, which every region got. Margin is roughly flat, so the rise held. I'd like the next price change to go out as a proper test, because this method costs us an afternoon and a wide interval, and a test would cost a fortnight and give a straight answer."*

Tools

Versions used for this chapter, checked in September 2026:

- **Python 3.12.3, pandas 3.0.2, numpy 2.4.4, scipy 1.17.1, statsmodels 0.15.0.** Nothing else is needed: DiD, matching, and RD are all ordinary regressions once the data is shaped correctly.
- **statsmodels** for the regressions, including `cov_type="cluster"` for clustered standard errors and `smf.logit` for propensity scores.
- **scipy.optimize** for the synthetic control weights.
- **Worth knowing about:** `linearmodels` (panel models and two-stage least squares), `DoWhy` and `EconML` (causal graphs, sensitivity analysis, and heterogeneous effects), and `CausalImpact` (a Bayesian time-series version of the synthetic-control idea). They automate the arithmetic, not the argument.
- **Companion files** in `ch31/`: `generate_ch31_data.py` (all three datasets, seed 31, with the true effects in its header) and `ch31_check.py` (checks the chapter's numbers).

The project: measure the regional price change

Goal: a causal estimate you can defend in a meeting, with its assumption stated and checked.

Option A: your own data. Any change that hit one group and not another, at a known date: a price change, a new opening hour, a policy in one branch, a product that shipped to one country first. Work on a copy, and keep customer data out of anything you share.

Option B: Riverstone. Estimate the effect of North's price rise on order volume, then extend it.

Steps:

1. **Write the counterfactual in one sentence** before touching the data: "North's order volume in October 2025 to June 2026 if prices had not risen."
2. **Compute both naive answers** (before-and-after, and treated-versus-untreated) and say in a line each why they're wrong here.
3. **Estimate the effect with DiD**, first as a two-by-two table, then as a regression with region and month fixed effects, with clustered standard errors.
4. **Check parallel trends:** the chart, and the quarter-by-quarter view.
5. **Run a placebo:** pretend the change happened in October 2024 and re-estimate. You should find nothing. If you find something, your design is picking up noise.
6. **Build a synthetic control** and compare it with the DiD estimate. Report the pre-change fit.
7. **Convert to money:** volume effect $\times$ average order value $\times$ price change, and say what it means for margin.
8. **Write four sentences** in section 31.9's shape: estimate, comparison, assumption and check, what would change your mind.

Deliverables: the analysis, the parallel-trends chart, the placebo result, and the four-sentence summary.

Stretch goals:

- Estimate the effect on revenue rather than volume, and explain why the two differ.
- Run the DiD dropping one comparison region at a time. How much does the estimate move?
- Use the QBR data to estimate the program's effect with matching, and separately with a DiD on 2024 versus 2025 revenue. Do the two agree?
- Design the randomized price test Meera asked for, using Chapter 30: unit, metric, MDE, sample size, duration.

You've got it when...

- ☐ I state the counterfactual in words before choosing a method.
- ☐ I can explain why before-and-after and treated-versus-untreated usually mislead, with an example.
- ☐ I can compute DiD as a table and as a regression, and say what the fixed effects absorb.
- ☐ I check parallel trends and show the chart, and I know what to do when they aren't parallel.
- ☐ I can fit a propensity model, match with a caliper, and report balance before and after.

- ☐ I know that matching assumes no unmeasured confounding, and I say so.
 - ☐ I can spot a threshold rule in a business process and use it for regression discontinuity, including the bunching check.
 - ☐ I understand that an RD estimate is local to the cut-off.
 - ☐ I can build a synthetic control and judge it by its pre-change fit.
 - ☐ I can state the three requirements for an instrument and explain why exclusion is usually the weak one.
 - ☐ Every causal claim I write carries its assumption and what would change my mind.
-

Recap

- A causal claim compares the world that happened with a **counterfactual** that didn't. Every method here is a way of building a stand-in for it.
 - **Before-and-after** is ruined by **confounding**; **treated-versus-untreated** is ruined by **selection**. In Riverstone's data they gave +1.7% and -23% for an effect that was -8%.
 - **Difference-in-differences** subtracts the untreated group's change. Work in logs, use **region and month fixed effects**, and **cluster** standard errors, remembering that few groups means wide, fragile intervals.
 - DiD rests on **parallel trends**: untestable for the future, checkable for the past. Show the chart and a period-by-period view.
 - **Matching** and **propensity scores** build comparison units one at a time. Use only pre-treatment variables, apply a **caliper**, report **balance** and how many units went unmatched, and state the **no unmeasured confounding** assumption.
 - **Regression discontinuity** uses a threshold rule as a natural experiment. Fit a line either side, measure the **jump**, check for **bunching**, report several **bandwidths**, and remember the answer is **local** to the cut-off.
 - **Synthetic control** blends untreated units to track the treated one before the change. Judge it by **pre-change fit**; infer with **placebo** units.
 - **Instrumental variables** need **relevance**, **exclusion**, and **independence**; only the first is testable, which is why good instruments are rare.
 - Evidence ranks roughly: experiment, then RD, then DiD and synthetic control, then matching, then naive comparisons. **Sensitivity analysis** asks how strong a missing confounder would have to be to overturn the result.
 - Write causal claims in four sentences: estimate with interval, comparison used, assumption and check, and what would change your mind.
-

Practice exercises

Work in `companion/ch31`, with the data built by `generate_ch31_data.py`. Predict each answer before running it.

Warm-up

1. Compute average monthly orders for each region, before and after October 2025. Which regions grew, and by how much?
2. Run the DiD on **revenue** instead of order volume. Why is the answer so different from the volume effect, and which one answers "did the price rise work"?
3. For the QBR program, compare the average 2024 revenue of participants and non-participants. What does that tell you before you estimate anything?
4. How many orders in the threshold dataset are within ₹500 of the cut-off, and what share of them got free delivery?

Core

5. Re-estimate the price effect using only West and South as comparison regions. Then only East. How much does the estimate move, and what does that tell you?
6. Run a **placebo test**: pretend the price rise happened on 1 October 2024 and estimate a DiD using only the months before the real change. What should you find, and what do you find?
7. Add clustered standard errors to the fixed-effects DiD (`cov_type="cluster"` grouped by region). How does the interval change, and why shouldn't you trust it too far?
8. Estimate the QBR effect with a DiD on the same customers (2024 versus 2025 revenue, participants versus not) instead of matching. Compare with section 31.5's estimates.
9. Redo the matching with a tighter caliper of 0.01 and with no caliper at all. Report pairs matched, balance, and the estimate for each.
10. Fit the RD with a quadratic on each side of the threshold at a bandwidth of ₹5,000. Does the estimate change? Which specification would you report?

Stretch

11. Run the synthetic-control **placebo**: build a synthetic West, South, and East in turn, and compare their post-October gaps with North's. Is North's gap unusual?
12. The RD dataset contains segment and region. Check that these are **smooth** across the threshold, and explain why that check matters.
13. Estimate the price effect with a regression that includes a region-specific linear trend. Does the effect survive?
14. Write the "what would change your mind" sentence for each of the three estimates in this chapter, and rank the three by how much you'd trust them.

Think about it (no code needed)

- l5. Riverstone's warehouse in Bhiwandi was expanded in March 2026, serving mostly West-region customers. How does that affect the price-rise DiD, and what would you do about it?
 - l6. A colleague proposes matching customers on their 2025 revenue to estimate the 2025 effect of the QBR program. What's wrong with that?
 - l7. Give one threshold rule in a business you know that could serve as a regression discontinuity, and one reason it might fail the bunching check.
 - l8. When would you prefer a weaker method with data you have over a stronger one that would take six months to arrange?
-

Key terms

causal claim · counterfactual · confounding · selection · treated and untreated groups · before-and-after · difference-in-differences · two-by-two table · fixed effects · clustered standard errors · parallel trends · event study · placebo test · matching · propensity score · caliper · matching without replacement · balance · standardized mean difference · overlap (common support) · no unmeasured confounding · regression discontinuity · running variable · cut-off · bandwidth · local effect · bunching · manipulation check · synthetic control · donor pool · pre-change fit · interference between units · instrumental variable · relevance · exclusion restriction · independence · two-stage least squares · sensitivity analysis

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 30, Inference & Experiments**, is the method to reach for whenever randomizing is possible; this chapter is what you do when it isn't.
 - **Chapter 22, Statistics Without Fooling Yourself**, is the intuition behind the warnings here.
 - **Chapter 43, Pricing & Revenue Analytics**, applies these methods to price changes, discounts, and elasticity.
 - **Chapter 19, Regression & Forecasting**, shares the machinery: the difference is entirely in what you claim from it.
 - **Chapter 55, Machine Learning in Production**, needs causal thinking to tell "the model is good" from "the model was given the easy cases".
 - **Chapter 73, Statistics, Probability & Experimentation Bank**, has causal-inference questions, including three "you can't A/B test this" cases.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
change = (panel.pivot_table(index="region", columns="is_after", values="orders", aggfunc="mean")
          .rename(columns={0: "before", 1: "after"}))
change["change_%"] = 100 * (change["after"] / change["before"] - 1)
print(change.round(1).to_string())
```

is_after	before	after	change_%
region			
East	1500.9	1625.4	8.3
North	2031.7	2067.1	1.7
South	2501.3	2697.9	7.9
West	3334.9	3725.3	11.7

Every region grew, North least of all. The gap between North's growth and the others' is the DiD estimate, seen in levels rather than logs.

2.

```
panel["log_revenue"] = np.log(panel["revenue"])
rev = smf.ols("log_revenue ~ treated:is_after + C(region) + C(month)", data=panel).fit()
vol = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
print(f"effect on volume: {100 * (np.exp(vol.params['treated:is_after']) - 1):+.1f}%")
print(f"effect on revenue: {100 * (np.exp(rev.params['treated:is_after']) - 1):+.1f}%")
print(f"price rise applied: +6.0%")
```

```
effect on volume: -7.0%
effect on revenue: +0.7%
price rise applied: +6.0%
```

Volume fell about 7%, but each remaining order is worth 6% more, so revenue is close to flat. Both are true and they answer different questions: the volume effect measures how customers responded, the revenue effect measures what it did to the top line. For "did the price rise work", the revenue (and ideally margin) effect is the one the board wants, with the volume effect as the explanation.

3.

```
print(qbr.groupby("in_qbr_program")[["revenue_2024", "growth_2024",
"years_as_customer"]].mean().round(3).to_string())
print(f"ratio of average 2024 revenue: {qbr.groupby('in_qbr_program')
['revenue_2024'].mean().iloc[1] / qbr.groupby('in_qbr_program')
['revenue_2024'].mean().iloc[0]:.2f}x")
```

	revenue_2024	growth_2024	years_as_customer
in_qbr_program			
0	755520.916	0.052	5.023
1	1225516.825	0.062	5.632
ratio of average 2024 revenue: 1.62x			

Participants were already 1.6 times the size of non-participants *before* the program began, and growing faster. Any raw comparison of 2025 revenue will mostly measure that. This table is the argument for matching, and it belongs at the top of the write-up.

4.

```
near = orders[orders["centred"].abs() <= 0.5]
print(f"orders within ₹500 of the cut-off: {len(near):,}")
print(f"share with free delivery: {near['free_delivery'].mean():.3f}")
print(f"reorder rate below: {near.loc[near['free_delivery'] == 0,
'repeat_within_90_days'].mean():.3f}")
print(f"reorder rate above: {near.loc[near['free_delivery'] == 1,
'repeat_within_90_days'].mean():.3f}")
```

```
orders within ₹500 of the cut-off: 5,686
share with free delivery: 0.502
reorder rate below: 0.328
reorder rate above: 0.412
```

Almost exactly half, which is what a smooth running variable around a threshold should give. It's also the cleanest comparison in the chapter: two groups of orders differing by a few hundred rupees and a delivery charge.

5.

```
for comparison in (["West", "South"], ["East"], ["West"], ["South"]):
    subset = panel[panel["region"].isin(["North"] + comparison)]
    fit = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=subset).fit()
    effect = fit.params["treated:is_after"]
    low, high = fit.conf_int().loc["treated:is_after"]
    print(f"comparison {str(comparison):22s}: {100 * (np.exp(effect) - 1):+5.1f}% "
          f"(CI {100 * (np.exp(low) - 1):+5.1f}% to {100 * (np.exp(high) - 1):+5.1f}%)")
```

```
comparison ['West', 'South']      : -7.5% (CI -10.9% to -3.8%)
comparison ['East']               : -6.2% (CI -9.8% to -2.4%)
comparison ['West']               : -9.1% (CI -13.1% to -4.9%)
comparison ['South']              : -5.8% (CI -10.0% to -1.4%)
```

The estimate moves by a couple of points depending on the comparison group, and every version is clearly negative. That spread is honest information: report the main specification, and say how much it moves when the comparison changes. An estimate that flips sign when you swap comparison regions is not a finding.

6.

```
placebo_data = panel[panel["month"] < "2025-10-01"].copy()
placebo_data["fake_after"] = (placebo_data["month"] >= "2025-01-01").astype(int)
placebo = smf.ols("log_orders ~ treated:fake_after + C(region) + C(month)",
data=placebo_data).fit()
effect = placebo.params["treated:fake_after"]
low, high = placebo.conf_int().loc["treated:fake_after"]
print(f"placebo effect: {100 * (np.exp(effect) - 1):+.1f}% "
      f"(95% CI {100 * (np.exp(low) - 1):+.1f}% to {100 * (np.exp(high) - 1):+.1f}%)")
print("a clean design finds nothing here")
```

```
placebo effect: +0.5% (95% CI -3.9% to +5.1%)
a clean design finds nothing here
```

The placebo effect is small and its interval comfortably includes zero, so the design isn't manufacturing effects out of noise. Run this before you trust any DiD: it costs three lines and catches designs that would "find" something whatever the data.

7.

```
clustered = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit(
    cov_type="cluster", cov_kwds={"groups": panel["region"]})
plain = smf.ols("log_orders ~ treated:is_after + C(region) + C(month)", data=panel).fit()
for name, fit in (("plain", plain), ("clustered", clustered)):
    low, high = fit.conf_int().loc["treated:is_after"]
    print(f"{name:10s}: effect {fit.params['treated:is_after']:+.4f}, "
          f"CI {low:+.4f} to {high:+.4f}, se {fit.bse['treated:is_after']:.4f}")
```

```
plain      : effect -0.0729, CI -0.1064 to -0.0394, se 0.0168
clustered  : effect -0.0729, CI -0.0978 to -0.0480, se 0.0127
```

Clustering changes the standard error, but with four clusters the formula behind it assumes a number of groups you don't have. Quote it, and add the caveat: with four regions, treat the interval as indicative. The fixes are a wild cluster bootstrap or randomization inference, and both belong in a write-up as a line saying which you used.

8.

```
long = pd.concat([
    qbr.assign(year=2024, log_revenue=qbr["log_2024"]),
    qbr.assign(year=2025, log_revenue=qbr["log_2025"])])
long["after"] = (long["year"] == 2025).astype(int)
did_qbr = smf.ols("log_revenue ~ in_qbr_program:after + C(customer_id) + C(year)", data=long).fit()
effect = did_qbr.params["in_qbr_program:after"]
print(f"DiD estimate: {100 * (np.exp(effect) - 1):+.1f}%")
print(f"matching:      {100 * (np.exp(matched_effect) - 1):+.1f}%")
print("true effect:  +7.0%")
```

```
DiD estimate: +9.0%
matching:     +7.1%
true effect:  +7.0%
```

The DiD uses each customer as their own control, which removes every fixed difference between accounts, measured or not. It lands close to the truth and it needs a weaker assumption than matching: not "we measured everything that drove selection", but "participants' revenue would have grown like non-participants' ". When you have a before period, prefer this.

9.

```
def match(caliper):
    available = qbr[qbr["in_qbr_program"] == 0].set_index("customer_id")["propensity"].to_dict()
    ids = []
    for _, row in qbr[qbr["in_qbr_program"] == 1].sort_values("propensity",
ascending=False).iterrows():
        if not available:
            break
        best = min(available, key=lambda cid: abs(available[cid] - row["propensity"]))
        if caliper is None or abs(available[best] - row["propensity"]) <= caliper:
            ids += [row["customer_id"], best]
            del available[best]
    subset = qbr[qbr["customer_id"].isin(ids)]
    effect = subset.groupby("in_qbr_program")["log_2025"].mean().diff().iloc[-1]
    return len(ids) // 2, smd(subset, "log_2024"), 100 * (np.exp(effect) - 1)

for caliper in (0.01, 0.05, None):
    pairs_n, balance, effect = match(caliper)
    print(f"caliper {str(caliper):>5}: {pairs_n:2d} pairs, balance on log_2024 {balance:+.3f},
effect {effect:+.1f}%")
print("true effect: +7.0%")
```

```
caliper 0.01: 56 pairs, balance on log_2024 -0.082, effect +0.4%
caliper 0.05: 64 pairs, balance on log_2024 +0.002, effect +7.1%
caliper None: 68 pairs, balance on log_2024 +0.071, effect +12.2%
true effect: +7.0%
```

Look at how much the answer moves: +0.4%, +7.1%, +12.2%, for the same data and the same method. With no caliper, every participant gets matched, including the largest accounts whose nearest "comparison" isn't close, and balance drifts. With a caliper of 0.01, only 56 pairs survive and the ones that do are a peculiar subset. The middle setting balances well and lands near the truth, but you couldn't have known that in advance without the answer key. This is the honest reason matching ranks below the other methods in section 31.9: several defensible choices give materially different answers, so report the choice, the balance, and how much the estimate moves when you change it.

10.

```
window = orders[orders["centred"].abs() <= 5]
linear = smf.ols("repeat_within_90_days ~ free_delivery + centred + free_delivery:centred",
data=window).fit()
quad = smf.ols("repeat_within_90_days ~ free_delivery + centred + I(centred ** 2) + "
               "free_delivery:centred + free_delivery:I(centred ** 2)", data=window).fit()
for name, fit in (("linear", linear), ("quadratic", quad)):
    low, high = fit.conf_int().loc["free_delivery"]
    print(f"{name:10s}: {100 * fit.params['free_delivery']:+.2f} points "
          f"(CI {100 * low:+.2f} to {100 * high:+.2f})")
print("true effect: +6.0 points")
```

```
linear      : +7.26 points (CI +5.61 to +8.90)
quadratic   : +8.31 points (CI +5.87 to +10.74)
true effect: +6.0 points
```

The quadratic sits about a point higher with a wider interval, and the two overlap comfortably, so the finding doesn't depend on the shape you fit. Report the linear fit at a narrow bandwidth as the main result and the others as robustness: high-order polynomials are known to produce strange jumps at

the boundary, and "we tried several and they agree" is a stronger sentence than any single specification.

11.

```
def synthetic_gap(target):
    donors = [r for r in ["West", "South", "East", "North"] if r != target]
    pre = wide_panel[wide_panel.index < "2025-10-01"]
    fit = minimize(lambda w: float(np.mean((pre[target].to_numpy() - pre[donors].to_numpy() @ w) **
2)),
                    np.repeat(1 / 3, 3), bounds=[(0, 1)] * 3,
                    constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}], method="SLSQP")
    blend = wide_panel[donors].to_numpy() @ fit.x
    after = wide_panel.index >= "2025-10-01"
    pre_fit = np.sqrt(np.mean((pre[target].to_numpy() - blend[:len(pre)]) ** 2))
    return pre_fit, (wide_panel[target].to_numpy() - blend)[after].mean()

for region in ["North", "West", "South", "East"]:
    pre_fit, gap = synthetic_gap(region)
    flag = " <-- the treated region" if region == "North" else ""
    print(f"{region:6s}: pre-change fit {pre_fit:.4f}, post-change gap {100 * (np.exp(gap) -
1):+5.1f}%{flag}")
```

```
North : pre-change fit 0.0429, post-change gap -6.6% <-- the treated region
West  : pre-change fit 0.2918, post-change gap +38.2%
South : pre-change fit 0.0415, post-change gap -0.0%
East  : pre-change fit 0.3074, post-change gap -21.4%
```

Read the fit column first. West and East can't be reproduced by a blend of the others at all (pre-change errors of 0.29 and 0.31 against North's 0.043), so their gaps are noise and tell you nothing. South fits as well as North does, and its post-change gap is zero. So the one usable placebo shows nothing while the treated region shows -6.6%. That's the informal significance test synthetic control uses, and with three donor regions the strongest statement available is "one in two", which is another way of saying four regions is a very small dataset.

12.

```
window = orders[orders["centred"].abs() <= 2].copy()
for column in ["segment", "region"]:
    share = pd.crosstab(window["free_delivery"], window[column], normalize="index")
    print(share.round(3).to_string())
    print()
```

segment	Hospitality	Retail	Wholesale
free_delivery			
0	0.338	0.326	0.336
1	0.332	0.336	0.332

region	East	North	South	West
free_delivery				
0	0.254	0.248	0.248	0.250
1	0.254	0.256	0.245	0.245

The mix of segments and regions is nearly identical either side of the cut-off. If it weren't, the two sides would differ in ways beyond the rule, and the jump could be that difference rather than the free delivery. Covariate smoothness is to RD what balance is to matching, and what the sample-ratio check is to an A/B test: the test of whether the comparison is fair.

13.

```
panel["t"] = ((panel["month"].dt.year - 2024) * 12 + panel["month"].dt.month)
trend = smf.ols("log_orders ~ treated:is_after + C(region) + C(month) + C(region):t",
data=panel).fit()
effect = trend.params["treated:is_after"]
low, high = trend.conf_int().loc["treated:is_after"]
print(f"with region-specific trends: {100 * (np.exp(effect) - 1):+.1f}% "
      f"(CI {100 * (np.exp(low) - 1):+.1f}% to {100 * (np.exp(high) - 1):+.1f}%)")
print("without them: -7.0%")
```

```
with region-specific trends: -4.8% (CI -10.4% to +1.2%)
without them: -7.0%
```

The effect shrinks to -4.8% and its interval now crosses zero. Don't read that as "the result failed": with 24 months and a change two-thirds of the way through, a region-specific trend fitted on the whole period soaks up much of the post-change drop, which is exactly the effect being measured. It is still a useful warning. The honest summary is: -7% in the main specification, -5% and not distinguishable from zero when each region is allowed its own trend, and the two are not far apart in business terms. Report both, and let the reader see the range.

14. One sentence each, ranked by how much they'd survive an argument:

- **Regression discontinuity (free delivery, +7 points):** *"This would change if customers could steer their order values across ₹25,000, which the density check says they don't."* Strongest, because the comparison group is almost mechanical, and weakest in scope: it speaks only for orders near the threshold.
- **Difference-in-differences (price rise, -7%):** *"This would change if something else hit North around October 2025, or if North was already drifting away from the other regions."* Strong on the evidence available, with the caveat that four regions is few.
- **Matching (quarterly reviews, +7%):** *"This would change if the sales team chose accounts using something we can't see, such as their sense of which accounts had potential."* Weakest, because that judgment is exactly the sort of thing that isn't in a table.

15. The expansion helps West-region customers from March 2026, which is *after* the price rise, so the comparison group's behavior changes partway through the post period. That inflates the other regions' growth and makes North look worse than it is. Three responses, in order of preference: end the analysis window in February 2026; drop West and use South and East as the comparison (exercise 5 shows the estimate holds); or add a control for the expansion period and report both. Whatever you choose, say it, because a reader who knows about the warehouse will ask.

16. Revenue in 2025 is measured *after* the program ran, so it's partly an effect of the thing being measured. Matching on it would pair participants with the non-participants who happened to reach the same revenue, throwing away the very difference the program caused, and biasing the estimate

toward zero. The rule is absolute: match only on what was known before treatment began, which here means 2024 revenue, 2024 growth, tenure, and segment.

17. Any rule with a sharp edge: free shipping above a value, a discount tier at a volume, a credit check above an order size, a bonus for hitting a monthly target, an SLA that applies above a ticket priority. Bunching is likely wherever the person affected can see the threshold and control the number: sales reps write orders at exactly the discount tier, expense claims come in just under the approval limit, and salespeople hit their target by a rupee in the last week of the month. If the histogram spikes at the edge, the rule is being gamed, and that spike is itself worth reporting to the business.

18. Almost always, when the decision is soon and reversible. A DiD that takes an afternoon and gives ± 3 points is worth more than a perfect experiment that reports after the decision is made. The reverse holds when the decision is expensive, hard to undo, or repeated: a pricing policy for every region, a system everyone will use for five years. The practical answer is usually both: measure what you can now with the weaker method, and design the experiment for the next change, which is exactly what Riverstone did after the price rise.